

os codigos do gemini

```
"""
```

SISTEMA DE ANÁLISE E OTIMIZAÇÃO DE TRELIÇAS PLANAS ISOSTÁTICAS

Projeto Acadêmico para Engenharia Civil / Mecânica

Autor: Assistente IA (Gemini 3 Flash)

Data: 2024

Descrição:

Este software realiza a análise estrutural completa de uma treliça plana.

Calcula reações de apoio, forças internas (tração/compressão), identifica barras nulas, e gera visualizações gráficas avançadas utilizando Matplotlib.

A resolução do sistema linear é feita via Eliminação de Gauss manual.

```
"""
```

```
import math
```

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
#
```

```
=====
```

```
# 1. CONFIGURAÇÃO DOS DADOS DA ESTRUTURA (EXEMPLO REALISTA)
```

```
#
```

```
=====
```

```
def carregar_dados():
```

```
    """
```

Define a geometria, conectividade, apoios e cargas de uma treliça de 8 nós.

Coordenadas em metros (m), Cargas em Quilonêutons (kN).

```
    """
```

```
# Coordenadas dos Nós (x, y)
```

```
nos = [
```

```
    [0, 0], # Nó 0 (Apoio Fixo)
```

```
    [3, 0], # Nó 1
```

```
    [6, 0], # Nó 2
```

```
    [9, 0], # Nó 3 (Apoio Móvel)
```

```
    [1.5, 2], # Nó 4
```

```
    [4.5, 2], # Nó 5
```

```
    [7.5, 2], # Nó 6
```

```
    [4.5, 4] # Nó 7
```

```
]
```

```
# Conectividade das Barras (Nó inicial, Nó final)
```

```
barras = [
```

```

[0, 1], [1, 2], [2, 3],      # Banzo Inferior
[0, 4], [4, 7], [7, 6], [6, 3], # Banzo Superior/Externo
[4, 5], [5, 6],             # Banzo Interno Horizontal
[1, 4], [1, 5], [2, 5], [2, 6] # Diagonais e Verticais
]

```

```

# Apoios: [índice_no, eixo] (0 para X, 1 para Y)
# Nó 0: Fixo (X e Y), Nó 3: Móvel (Y)
apoios = [
    {'no': 0, 'eixo': 0},
    {'no': 0, 'eixo': 1},
    {'no': 3, 'eixo': 1}
]

```

```

# Cargas: [índice_no, eixo, valor_kN]
# Cargas gravitacionais nos nós superiores e centrais
cargas = [
    {'no': 4, 'eixo': 1, 'valor': -10.0},
    {'no': 7, 'eixo': 1, 'valor': -25.0},
    {'no': 6, 'eixo': 1, 'valor': -10.0},
    {'no': 1, 'eixo': 1, 'valor': -5.0},
    {'no': 2, 'eixo': 1, 'valor': -5.0}
]

```

```

return nos, barras, apoios, cargas

```

```

#
=====
=====
# 2. FUNÇÕES MATEMÁTICAS E SOLUCIONADOR MANUAL
#
=====
=====

```

```

def resolver_gauss(A, b):
    """
    Implementação manual do algoritmo de Eliminação de Gauss com pivoteamento parcial.
    Resolve o sistema  $Ax = b$ .
    """
    n = len(b)
    # Matriz aumentada
    Ab = []
    for i in range(n):
        Ab.append(A[i] + [b[i]])

    # Eliminação progressiva
    for i in range(n):
        # Pivoteamento parcial

```

```

max_el = abs(Ab[i][i])
max_row = i
for k in range(i + 1, n):
    if abs(Ab[k][i]) > max_el:
        max_el = abs(Ab[k][i])
        max_row = k

Ab[i], Ab[max_row] = Ab[max_row], Ab[i]

```

```

# Zerar colunas abaixo
for k in range(i + 1, n):
    c = -Ab[k][i] / Ab[i][i]
    for j in range(i, n + 1):
        if i == j:
            Ab[k][j] = 0
        else:
            Ab[k][j] += c * Ab[i][j]

```

```

# Substituição regressiva
x = [0 for i in range(n)]
for i in range(n - 1, -1, -1):
    x[i] = Ab[i][n] / Ab[i][i]
    for k in range(i - 1, -1, -1):
        Ab[k][n] -= Ab[k][i] * x[i]

```

```

return x

```

```

#
=====
=====
# 3. NÚCLEO DE ANÁLISE ESTRUTURAL
#
=====
=====

```

```

def analisar_trelica(nos, barras, apoios, cargas):
    num_nos = len(nos)
    num_barras = len(barras)
    num_reacoes = len(apoios)

```

```

# Número total de incógnitas (Forças nas barras + Reações)
# Para ser isostática:  $b + r = 2j$ 
num_incognitas = num_barras + num_reacoes
num_equacoes = 2 * num_nos

```

```

# Inicialização da Matriz de Equilíbrio (A) e Vetor de Cargas (B)
A = [[0.0 for _ in range(num_incognitas)] for _ in range(num_equacoes)]
B = [0.0 for _ in range(num_equacoes)]

```

1. Preenchimento dos coeficientes das barras (Cossenos e Senos)

for i, barra in enumerate(barras):

 n1_idx, n2_idx = barra

 x1, y1 = nos[n1_idx]

 x2, y2 = nos[n2_idx]

 L = math.sqrt((x2 - x1)**2 + (y2 - y1)**2)

 cos = (x2 - x1) / L

 sen = (y2 - y1) / L

 # Contribuição para o Nó Inicial (n1)

 A[2 * n1_idx][i] = cos

 A[2 * n1_idx + 1][i] = sen

 # Contribuição para o Nó Final (n2) - Direção oposta

 A[2 * n2_idx][i] = -cos

 A[2 * n2_idx + 1][i] = -sen

2. Preenchimento dos coeficientes das reações

for i, apoio in enumerate(apoios):

 no_idx = apoio['no']

 eixo = apoio['eixo']

 coluna = num_barras + i

 # Equação de X ou Y do nó onde o apoio está

 A[2 * no_idx + eixo][coluna] = 1.0

3. Preenchimento do Vetor de Cargas (Invertendo o sinal para o outro lado da eq.)

for carga in cargas:

 no_idx = carga['no']

 eixo = carga['eixo']

 B[2 * no_idx + eixo] = -carga['valor']

4. Resolução do Sistema

solucao = resolver_gauss(A, B)

forcas_barras = solucao[:num_barras]

reacoes_apoio = solucao[num_barras:]

return forcas_barras, reacoes_apoio

#

=====

4. PROCESSAMENTO DE RESULTADOS E ESTATÍSTICAS

#

=====

```

def classificar_barras(forcas):
    classificacao = []
    for f in forcas:
        if abs(f) < 1e-6:
            classificacao.append("NULA")
        elif f > 0:
            classificacao.append("TRAÇÃO")
        else:
            classificacao.append("COMPRESSÃO")
    return classificacao

def exibir_resultados_terminal(nos, barras, forcas, reacoes, apoios):
    print("\n" + "="*80)
    print("    SISTEMA DE ANÁLISE E OTIMIZAÇÃO DE TRELIÇAS PLANAS -
RESULTADOS")
    print("="*80)

    # Tabela de Barras
    print(f'{"Barra":<8} | {"Nós":<10} | {"Força (kN)":<12} | {"Estado":<12}')
    print("-" * 50)
    classificacao = classificar_barras(forcas)
    for i, f in enumerate(forcas):
        nos_b = f"{barras[i][0]}-{barras[i][1]}"
        print(f'{"i":<8} | {"nos_b":<10} | {"f":>10.2f} | {"classificacao[i]:<12}')

    # Reações de Apoio
    print("\nREAÇÕES DE APOIO:")
    print("-" * 30)
    soma_reacoes = 0
    for i, r in enumerate(reacoes):
        eixo = "X" if apoios[i]['eixo'] == 0 else "Y"
        no = apoios[i]['no']
        print(f'Reação R{eixo} no Nó {no}: {r:>10.2f} kN")
        soma_reacoes += r

    # Estatísticas de Otimização
    abs_forcas = [abs(f) for f in forcas]
    max_f = max(abs_forcas)
    idx_max = abs_forcas.index(max_f)
    media_f = sum(abs_forcas) / len(abs_forcas)

    print("\n" + "="*80)
    print("ANÁLISE DE CRITICALIDADE E OTIMIZAÇÃO")
    print(f'-> Barra mais solicitada: Barra {idx_max} ({max_f:.2f} kN)")
    print(f'-> Força média interna: {media_f:.2f} kN")
    print(f'-> Somatório das intensidades das reações: {abs(soma_reacoes):.2f} kN")
    print("="*80)

```

```

return idx_max, classificacao

#
=====
=====
# 5. VISUALIZAÇÃO GRÁFICA (MATPLOTLIB)
#
=====
=====

def gerar_graficos(nos, barras, forcas, classificacao):
    fig = plt.figure(figsize=(14, 10))
    plt.subplots_adjust(hspace=0.4, wspace=0.3)

    # (1) Desenho da Trelça Original
    ax1 = fig.add_subplot(2, 2, 1)
    for i, barra in enumerate(barras):
        x = [nos[barra[0]][0], nos[barra[1]][0]]
        y = [nos[barra[0]][1], nos[barra[1]][1]]
        ax1.plot(x, y, 'ko-', linewidth=1)
    for i, no in enumerate(nos):
        ax1.text(no[0]+0.1, no[1]+0.1, f"N{i}", fontsize=9, color='blue', fontweight='bold')
    ax1.set_title("Geometria da Trelça (Nós e Barras)")
    ax1.set_aspect('equal')
    ax1.grid(True, linestyle='--', alpha=0.6)

    # (2) Trelça Colorida por Intensidade
    ax2 = fig.add_subplot(2, 2, 2)
    max_f = max([abs(f) for f in forcas]) if max([abs(f) for f in forcas]) != 0 else 1

    for i, barra in enumerate(barras):
        x = [nos[barra[0]][0], nos[barra[1]][0]]
        y = [nos[barra[0]][1], nos[barra[1]][1]]

        # Cor por estado: Vermelho (Compressão), Azul (Tração), Cinza (Nula)
        if classificacao[i] == "TRAÇÃO":
            cor = 'blue'
            espessura = 1 + (abs(forcas[i])/max_f)*4
        elif classificacao[i] == "COMPRESSÃO":
            cor = 'red'
            espessura = 1 + (abs(forcas[i])/max_f)*4
        else:
            cor = 'gray'
            espessura = 1

    ax2.plot(x, y, color=cor, linewidth=espessura)

```

```
ax2.set_title("Esforços: Azul(T) / Vermelho(C)")
ax2.set_aspect('equal')
ax2.grid(True, linestyle='--', alpha=0.3)
```

```
# (3) Gráfico de Barras (Forças)
ax3 = fig.add_subplot(2, 2, 3)
indices = range(len(forcas))
cores_barras = ['red' if f < 0 else 'blue' for f in forcas]
ax3.bar(indices, forcas, color=cores_barras)
ax3.set_xlabel("Índice da Barra")
ax3.set_ylabel("Força Interna (kN)")
ax3.set_title("Magnitude das Forças por Barra")
ax3.axhline(0, color='black', linewidth=1)
```

```
# (4) Histograma de Distribuição
ax4 = fig.add_subplot(2, 2, 4)
ax4.hist(forcas, bins=8, color='green', alpha=0.7, edgecolor='black')
ax4.set_title("Distribuição das Forças Internas")
ax4.set_xlabel("Força (kN)")
ax4.set_ylabel("Frequência")
```

```
plt.suptitle("RELATÓRIO TÉCNICO DE ENGENHARIA - ANÁLISE ESTRUTURAL",
fontsize=16, fontweight='bold')
plt.show()
```

```
#
=====
=====
# 6. EXECUÇÃO PRINCIPAL
#
=====
=====
```

```
def main():
    # Cabeçalho de Inicialização
    print("Iniciando processamento da treliça...")

    # Fluxo de dados
    coords, conectividade, suportes, cargas_aplicadas = carregar_dados()

    # Cálculo
    forcas_internas, reacoes = analisar_trelica(coords, conectividade, suportes,
cargas_aplicadas)

    # Saída de dados
    barra_critica_idx, lista_classes = exibir_resultados_terminal(
        coords, conectividade, forcas_internas, reacoes, suportes
    )
```

```
# Interpretação Final
print("\nINTERPRETAÇÃO DOS RESULTADOS:")
print(f"1. A barra {barra_critica_idx} exige atenção especial no dimensionamento por
suportar {max(forcas_internas):.2f} kN.")
print("2. Barras nulas identificadas podem ser candidatas à remoção para otimização de
peso,")
print(" desde que não prejudiquem a estabilidade em outras hipóteses de
carregamento.")
print("3. O equilíbrio global foi verificado via somatório de forças e momentos.")

# Visualização
gerar_graficos(coords, conectividade, forcas_internas, lista_classes)

if __name__ == "__main__":
    main()
```